ANALYSE PRÉLIMINAIRE — APPRENTISSAGE PAR RENFORCEMENT SUR *FrozenLake*

A PREPRINT

Ibrahim R. HADJ-ARAB
Faculté des sciences et de génie
Université Laval
ibrahim-rabah.hadj-arab.1@ulaval.ca

Khalil EN NOUAARY
Faculté des sciences et de génie
Université Laval
khalil.en-nouaary.1@ulaval.ca

30 mars 2026

Code source disponible à : `https://github.com/Ribraba/IFT-7201-Projet`

1 Cas d'étude

1.1 Composantes de l'environnement

FrozenLake-v1 [Towers et al., 2024] est un environnement de navigation en grille $N \times N$. L'espace d'états est l'ensemble des N^2 cases ; l'espace d'actions contient quatre déplacements (haut, bas, gauche, droite). Trois composantes peuvent être manipulées pour moduler la difficulté de l'environnement et l'exposition aux situations dangereuses :

- **Degré de stochasticité.** Le paramètre `is_slippery` contrôle la dynamique de transition. Lorsque activé, chaque action entraîne le déplacement souhaité avec probabilité $1/3$ et l'un des deux déplacements perpendiculaires avec probabilité $1/3$ chacun, rendant l'environnement non-déterministe.
- **Nombre de situations dangereuses.** Les cases de type `H` (trous) constituent les situations dangereuses. Augmenter leur nombre réduit la surface praticable et multiplie les risques de chute.
- **Positionnement des situations dangereuses.** Regrouper les trous de façon contiguë (ex. rangée horizontale) crée un obstacle de type falaise, forçant l'agent à choisir entre un chemin court risqué et un détour sûr.

La fonction de récompense est mise en forme pour accélérer la propagation du signal de danger : chute dans un trou ($r = -1{,}0$), atteinte de l'objectif ($r = +1{,}0$), collision avec un mur ($r = -0{,}1$), déplacement ordinaire ($r = 0{,}0$).

1.2 Cas d'étude

Le Tableau 1 et la Figure 1 présentent les trois configurations retenues.

TABLE 1 – Configurations des trois cas d'étude.

	Facile	**Moyen**	**Difficile**
Taille de la grille	4×4	4×4	6×6
`is_slippery`	Non	Oui	Oui
Nombre de trous	3	3	4
Positionnement	Colonne droite	Colonne droite	Rangée centrale

Facile (4×4, déterministe, 3 trous en colonne). Les transitions sont déterministes. Ce cas sert de référence : en l'absence de bruit, toute stratégie TD doit converger vers un taux de chutes nul. Il valide l'implémentation et établit une borne supérieure de performance.

Moyen (4×4, stochastique, 3 trous en colonne). Même topologie que *Facile*, mais avec `is_slippery=True`. Ce cas isole l'effet de la stochasticité : les glissades aléatoires peuvent précipiter l'agent vers un trou même si son action choisie était sûre, quantifiant ainsi l'impact du non-déterminisme sur la sécurité durant l'apprentissage.

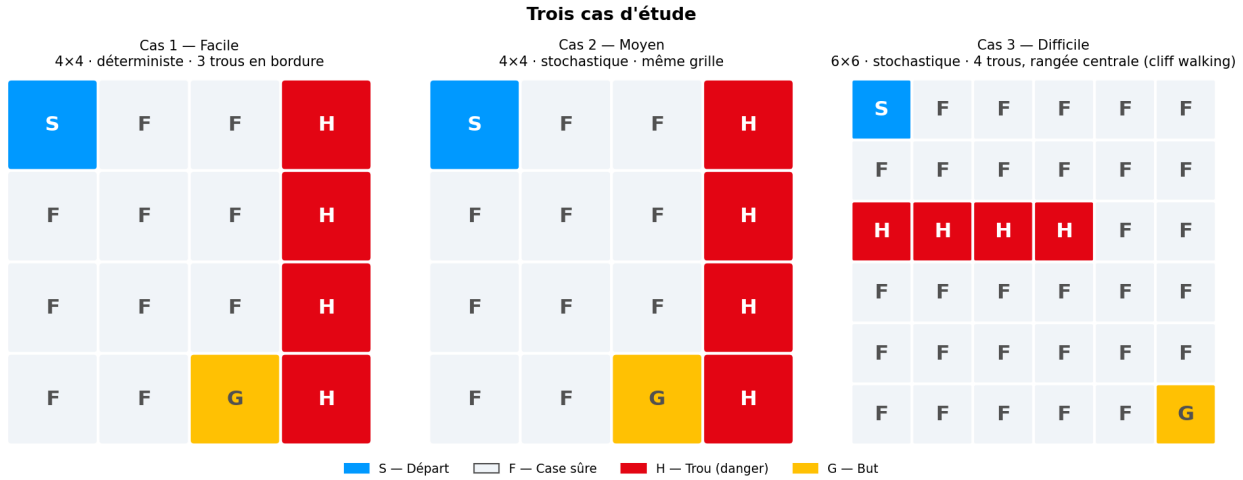


FIGURE 1 – Visualisation des trois environnements. `S` : départ, `G` : objectif, `H` : trou (situation dangereuse), `F` : case libre.

Difficile (6×6, stochastique, 4 trous contigus en rangée centrale). La rangée `HHHHFF` crée une barrière partielle analogue au problème du *cliff walking* [Sutton and Barto, 2018, chap. 6] : il existe un chemin court mais risqué longeant les trous et un long chemin sûr par les colonnes de droite. Ce cas combine les effets de la stochasticité, d'un nombre accru de trous et d'un positionnement hostile, poussant les deux stratégies à exhiber des comportements divergents face au risque.

2 Stratégies considérées

Deux stratégies de différence temporelle (TD) sont sélectionnées comme baselines : Q-learning et SARSA. Ces méthodes ne disposent d'aucun mécanisme conçu explicitement pour éviter les situations dangereuses, ce qui permet d'établir une limite de référence que toute stratégie de sécurité ultérieure devra dépasser. Le choix de méthodes tabulaires (plutôt que DQN [Mnih et al., 2015] ou PPO) est justifié par la taille réduite de l'espace d'états ($4 \times 4 = 16$ et $6 \times 6 = 36$ états) : les méthodes tabulaires convergent vers la solution exacte sans approximation de fonction, ce qui garantit que les différences observées reflètent bien les propriétés intrinsèques des algorithmes. De plus, Q-learning et SARSA forment la paire canonique *off-policy* / *on-policy* dont les comportements divergents face aux obstacles sont directement prédits par la théorie [Sutton and Barto, 2018].

Q-learning [Watkins and Dayan, 1992] est une méthode *off-policy* dont la règle de mise à jour repose sur la valeur maximale de l'état suivant :

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]. \tag{1}$$

Cette mise à jour optimiste suppose que la politique optimale sera suivie dans le futur, indépendamment de l'exploration en cours. Dans des environnements à obstacles groupés, cette hypothèse peut conduire à sous-estimer le risque réel de chute et à préférer le chemin court risqué.

SARSA [Rummery and Niranjan, 1994] est une méthode *on-policy* qui utilise l'action réellement sélectionnée selon la politique ε-greedy courante :

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma\, Q(s',a') - Q(s,a)\big]. \tag{2}$$

Puisque a' peut être une action aléatoire (avec probabilité ε), SARSA intègre le coût de l'exploration dans ses estimations. On s'attend donc à ce qu'il apprenne à éviter davantage les cases proches des trous, au prix d'une propagation plus lente du signal de danger en début d'apprentissage. La confrontation de ces deux comportements opposés face au risque est la principale motivation de leur sélection.

3 Méthodologie expérimentale

Librairies. L'environnement est fourni par Gymnasium 1.2.3 [Towers et al., 2024]. Q-learning et SARSA sont implémentés directement en Python/NumPy 2.2.6, sans recours à Stable-Baselines3 [Raffin et al., 2021], afin de contrôler précisément les mises à jour TD.

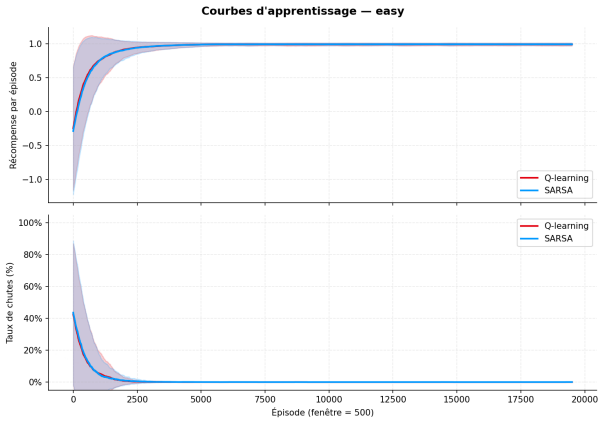
Hyper-paramètres. Les deux algorithmes partagent : $\gamma = 0{,}99$ (horizon long, adapté aux trajectoires multi-étapes jusqu'à l'objectif), $\alpha_0 = 0{,}5$ (valeur standard pour les méthodes TD tabulaires [Sutton and Barto, 2018]), décroissance $0{,}9995$ pour α et ε jusqu'aux minima $\alpha_{\min} = 0{,}01$ et $\varepsilon_{\min} = 0{,}05$. Ce taux de décroissance amène ε à son minimum après $\approx 6\,000$ épisodes, assurant une phase d'exploration large suivie d'une exploitation progressive sur les 14 000 épisodes restants.

Paramètres expérimentaux. Chaque configuration est entraînée sur 20 000 épisodes, avec un maximum de 200 pas par épisode. Les expériences sont répétées avec $n = 10$ graines indépendantes (graines 0 à 9) pour produire des intervalles de confiance (moyenne $\pm$ écart-type). L'évaluation finale utilise la politique greedy ($\varepsilon = 0$) sur 500 épisodes.

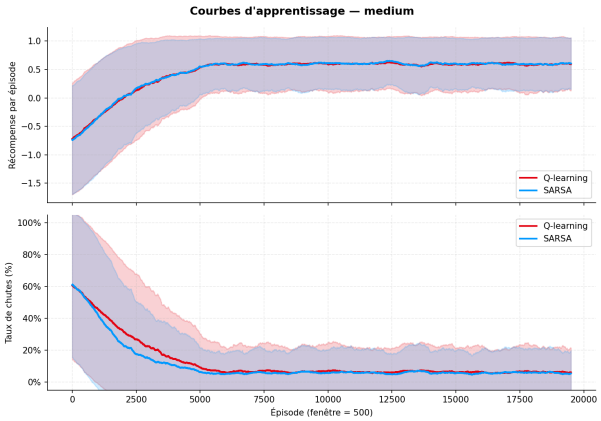
Indicateurs de performance. L'étude vise à évaluer la sécurité *durant* l'apprentissage et non uniquement sur la politique finale. Quatre indicateurs sont mesurés :

- **Taux de chutes durant l'entraînement** : fraction d'épisodes terminés dans un trou (lissé sur 500 épisodes). Mesure la sécurité au cours de l'apprentissage.
- **Récompense cumulée durant l'entraînement** : indicateur de convergence de la politique.
- **Taux d'états dangereux à l'évaluation** : fraction de pas de temps passés dans une case adjacente à un trou lors de l'évaluation greedy. Quantifie le risque de proximité avec les obstacles de la politique finale.
- **Taux de chemins sécurisés à l'évaluation** : fraction des épisodes réussis dont la trajectoire chevauche à au moins 50 % le chemin optimal de sécurité, calculé par une variante de l'algorithme de Dijkstra [Dijkstra, 1959] pénalisant les cases adjacentes à un trou. Quantifie la capacité de la politique finale à emprunter le détour sûr plutôt que le chemin court risqué.

4 Résultats



(a) **Facile** — Convergence vers 0% de chutes dès $\approx 2\,000$ épisodes. Q-learning converge légèrement plus vite grâce à ses mises à jour optimistes. Valide les implémentations en environnement déterministe.



(b) **Moyen** — La stochasticité fait bondir le taux de chutes initial à $\approx 61\%$ (contre $\approx 43\%$ en Facile). Le taux ne converge pas vers zéro : $\varepsilon_{\min} = 5\%$ combiné aux glissades maintient un risque irréductible.

FIGURE 2 – Courbes d'apprentissage — cas Facile et Moyen. La bande colorée représente $\pm$ un écart-type sur 10 répétitions.

Le Tableau 2 synthétise les indicateurs d'évaluation. Sur le cas Difficile, l'écart de boucles est frappant : Q-learning accumule $15{,}3\% \pm 7{,}7\%$ de *timeouts* contre seulement $3{,}1\% \pm 1{,}7\%$ pour SARSA. SARSA réduit également la proximité aux trous ($27{,}4\%$ contre $32{,}8\%$) et emprunte davantage le chemin sûr identifié par Dijkstra [Dijkstra, 1959] ($66{,}3\% \pm 2{,}5\%$ contre $63{,}7\% \pm 4{,}2\%$), avec une variance moindre confirmant une plus grande cohérence.

Ces résultats convergent : Q-learning converge plus rapidement mais adopte une politique moins stable en environnement stochastique à obstacles groupés, tandis que SARSA réduit les boucles, la proximité aux trous et préfère davantage le chemin sûr. Ces limitations motivent l'introduction d'un mécanisme de sécurité explicite dans le rapport final.

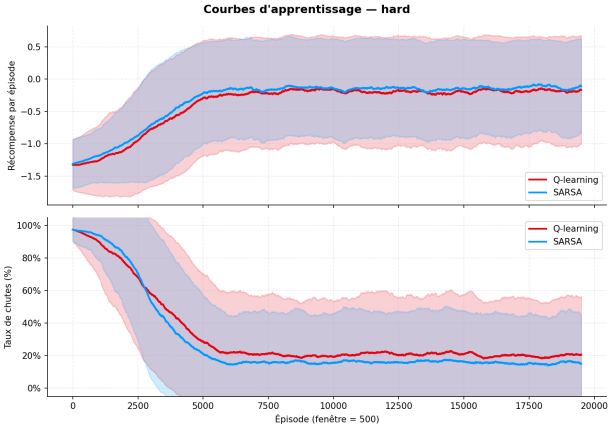
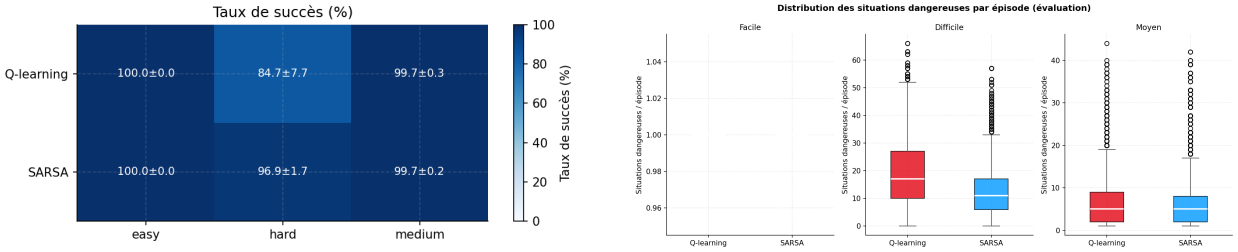


FIGURE 3 – **Difficile** — Les deux stratégies démarrent à un taux de chutes similaire ($\approx 97\%$), mais divergent en fin d'apprentissage, reproduisant le phénomène de *cliff walking* [Sutton and Barto, 2018] : SARSA converge vers $14{,}9\%$ de chutes contre $20{,}6\%$ pour Q-learning. La mise à jour *on-policy* intègre le coût de l'exploration et pousse l'agent vers le détour sûr.



(a) Taux de succès à l'évaluation (politique greedy, 500 épisodes). Les échecs sont exclusivement des *timeouts* — le taux de chutes est nul.

(b) Distribution des situations dangereuses par épisode. Sur le cas Difficile, SARSA affiche une médiane plus basse et une variance moindre.

FIGURE 4 – Évaluation finale — comparaison Q-learning vs SARSA.

TABLE 2 – Indicateurs d'évaluation finale (politique greedy, 500 épisodes, moyenne $\pm$ écart-type, 10 répétitions). Chemin sûr : overlap $\geq 50\%$ avec le chemin Dijkstra (cas Difficile uniquement).

		Facile	**Moyen**	**Difficile**
Q-learning	Succès	$100{,}0 \pm 0{,}0\%$	$99{,}7 \pm 0{,}3\%$	$84{,}7 \pm 7{,}7\%$
	Boucles	$0{,}0 \pm 0{,}0\%$	$0{,}3 \pm 0{,}3\%$	$15{,}3 \pm 7{,}7\%$
	Danger	$20{,}0 \pm 0{,}0\%$	$28{,}5 \pm 1{,}6\%$	$32{,}8 \pm 1{,}1\%$
	Chemin sûr	—	—	$63{,}7 \pm 4{,}2\%$
SARSA	Succès	$100{,}0 \pm 0{,}0\%$	$99{,}7 \pm 0{,}2\%$	$96{,}9 \pm 1{,}7\%$
	Boucles	$0{,}0 \pm 0{,}0\%$	$0{,}3 \pm 0{,}2\%$	$3{,}1 \pm 1{,}7\%$
	Danger	$20{,}0 \pm 0{,}0\%$	$27{,}7 \pm 0{,}6\%$	$27{,}4 \pm 1{,}4\%$
	Chemin sûr	—	—	$66{,}3 \pm 2{,}5\%$

Références

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium : A standard interface for reinforcement learning environments. *arXiv preprint arXiv :2407.17032*, 2024.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning : An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540) :529–533, 2015.

Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4) :279–292, 1992.

Gavin A. Rummery and Mahesan Niranjan. On-line q-learning using connectionist systems. Technical Report CUED/F-INFENG/TR 166, University of Cambridge, 1994.

Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-Baselines3 : Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268) :1–8, 2021.

Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1 :269–271, 1959.